

Modern Computing and Python Programming

(26ESCC106)

Manjunath Prasad H. R.

Assistant Professor

Department of Artificial Intelligence and Machine Learning

Mangalore Institute of Technology and Engineering



- 1 Course Description
- 2 Course Module Topics
 - Module Topics
 - Module Lab Component
- 3 Course Outcomes
- 4 Textbooks
 - Reference Books and Web Links
- 5 Module 1: Scalable Computing over the Internet
 - The Age of Internet Computing
 - Scalable Computing Trends and New Paradigms
 - Summary

Course at a Glance

Semester	I / II	CIE Marks	50
Course Code	26ESCC106	SEE Marks	50
Teaching Hrs./Week (L:T:P:S)	3:0:2:0	Exam Hrs.	03
Total Hrs	64 (40L + 24P)	Credits	04

- ➊ An **introductory course** on modern computing approaches, Python programming, and Artificial Intelligence & Machine Learning.
- ➋ Begins with the **evolution of computing** and contemporary paradigms — parallel, distributed, cluster, grid, cloud, and edge computing.
- ➌ Covers essential **Python programming constructs**, problem-solving techniques, and file processing.
- ➍ Develops basic applications using appropriate **Python libraries** (NumPy, Pandas, Scikit-learn).
- ➎ Concludes with an overview of **Artificial Intelligence**, intelligent agents, and the basic **Machine Learning workflow**.

- ➊ An **introductory course** on modern computing approaches, Python programming, and Artificial Intelligence & Machine Learning.
- ➋ Begins with the **evolution of computing** and contemporary paradigms — parallel, distributed, cluster, grid, cloud, and edge computing.
- ➌ Covers essential Python programming constructs, problem-solving techniques, and file processing.
- ➍ Develops basic applications using appropriate Python libraries (NumPy, Pandas, Scikit-learn).
- ➎ Concludes with an overview of Artificial Intelligence, intelligent agents, and the basic Machine Learning workflow.

- ➊ An **introductory course** on modern computing approaches, Python programming, and Artificial Intelligence & Machine Learning.
- ➋ Begins with the **evolution of computing** and contemporary paradigms — parallel, distributed, cluster, grid, cloud, and edge computing.
- ➌ Covers essential **Python programming constructs**, problem-solving techniques, and file processing.
- ➍ Develops basic applications using appropriate **Python libraries** (NumPy, Pandas, Scikit-learn).
- ➎ Concludes with an overview of **Artificial Intelligence**, intelligent agents, and the basic **Machine Learning workflow**.

- ➊ An **introductory course** on modern computing approaches, Python programming, and Artificial Intelligence & Machine Learning.
- ➋ Begins with the **evolution of computing** and contemporary paradigms — parallel, distributed, cluster, grid, cloud, and edge computing.
- ➌ Covers essential **Python programming constructs**, problem-solving techniques, and file processing.
- ➍ Develops basic applications using appropriate **Python libraries** (NumPy, Pandas, Scikit-learn).
- ➎ Concludes with an overview of **Artificial Intelligence**, intelligent agents, and the basic **Machine Learning workflow**.

- ➊ An **introductory course** on modern computing approaches, Python programming, and Artificial Intelligence & Machine Learning.
- ➋ Begins with the **evolution of computing** and contemporary paradigms — parallel, distributed, cluster, grid, cloud, and edge computing.
- ➌ Covers essential **Python programming constructs**, problem-solving techniques, and file processing.
- ➍ Develops basic applications using appropriate **Python libraries** (NumPy, Pandas, Scikit-learn).
- ➎ Concludes with an overview of **Artificial Intelligence**, intelligent agents, and the basic **Machine Learning workflow**.

This course is designed to:

- 1 Impart knowledge on **modern computing approaches** including parallel, distributed, cluster, grid, cloud, and edge computing.
- 2 Impart knowledge on Python data types, operators, and input/output statements.
- 3 Provide knowledge of strings, control structures, built-in data structures, functions, and file handling in Python.
- 4 Provide fundamentals of artificial intelligence and machine learning techniques.

This course is designed to:

- 1 Impart knowledge on **modern computing approaches** including parallel, distributed, cluster, grid, cloud, and edge computing.
- 2 Impart knowledge on Python **data types, operators**, and **input/output** statements.
- 3 Provide knowledge of **strings, control structures, built-in data structures, functions**, and **file handling** in Python.
- 4 Provide fundamentals of **artificial intelligence and machine learning** techniques.

This course is designed to:

- 1 Impart knowledge on **modern computing approaches** including parallel, distributed, cluster, grid, cloud, and edge computing.
- 2 Impart knowledge on Python **data types, operators**, and **input/output** statements.
- 3 Provide knowledge of **strings, control structures, built-in data structures, functions**, and **file handling** in Python.
- 4 Provide fundamentals of **artificial intelligence and machine learning** techniques.

This course is designed to:

- ➊ Impart knowledge on **modern computing approaches** including parallel, distributed, cluster, grid, cloud, and edge computing.
- ➋ Impart knowledge on Python **data types, operators**, and **input/output** statements.
- ➌ Provide knowledge of **strings, control structures, built-in data structures, functions**, and **file handling** in Python.
- ➍ Provide fundamentals of **artificial intelligence** and **machine learning** techniques.

Course Module 1 Topics - Modern Computing Approaches (8L)

- ♣ **Evolution of Computing:** Scalable computing over the Internet, the Age of Internet Computing, Scalable Computing Trends and New Paradigms
- ♣ **Parallel and Distributed Computing:** Eras of computing, parallel vs. distributed computing, parallel processing, elements of distributed computing
- ♣ **Cluster Computing:** Introduction, why clusters exist, components and characteristics, types, examples
- ♣ **Grid Computing:** Definition, power grid analogy, virtual organisation, characteristics, types, examples, advantages and limitations
- ♣ **Cloud Computing:** Introduction, cloud reference model (IaaS, PaaS, SaaS), types of clouds (public, private, hybrid, community)
- ♣ **Edge Computing:** Introduction, need, architecture, cloud vs. edge, applications

Course Module 2 Topics - Python Fundamentals (8L + 6P)

- ♣ About Python, Program Structure
- ♣ Variables, Keywords, Identifiers, Literals
- ♣ Input and Output Statements
- ♣ Data Types, Operators, Expressions
- ♣ Operator Precedence, Type Conversion
- ♣ Basic Debugging Techniques

Module 2 Laboratory - Python Basics

- ♣ Read student name, USN, and marks of three courses; compute total and average; display in a formatted way.
- ♣ Read the radius of a circle; compute area and circumference; convert the area to an integer.
- ♣ Accept an integer, a float, a string, and a Boolean; display values and data types; perform arithmetic and string operations.
- ♣ Accept two numbers and perform addition, multiplication, division, floor division, modulus, and exponentiation.

Course Module 3 Topics - Strings, Flow Control, and Functions (8L + 6P)

- ♣ **Strings:** Creating and storing strings, built-in string functions, string operators, indexing & slicing, formatting (f-strings & `format()`)
- ♣ **Flow Control:** `if`, `if-else`, `elif`; `for`, `while`; nested loops; `break`, `continue`, `pass`
- ♣ **Functions:** Defining & calling, built-in and user-defined functions, arguments & parameters, global and local variables, command line arguments, lambda functions, importing user-defined modules

Module 3 Laboratory - Strings, Menus, and Functions

- ♣ String operations: indexing, slicing, case conversion, counting, replacing, split/join, startswith/endswith.
- ♣ Menu-driven program: even/odd, largest of three, multiplication table, primes in a range, exit.
- ♣ Factorial using a function; simple interest with default arguments; square using a lambda.
- ♣ Create a module `mymath.py` with arithmetic functions and import it into another program.

- ♣ **Python Data Structures:** Mutable and immutable objects, lists, tuples, sets, dictionaries
- ♣ Common operations on lists, tuples, and dictionaries
- ♣ Passing lists, tuples, and dictionaries to functions
- ♣ **File Handling:** Introduction to files, opening a file, file modes, reading, writing, appending, closing a file

Module 4 Laboratory - Data Structures and Files

- ♣ List operations: insert, delete, sort, reverse; tuple of names: display, count, index.
- ♣ Dictionary of USN → name: add, update, delete, display; functions accepting list, tuple, and dictionary.
- ♣ Text files: write, append, read; count characters, words, and lines; `read()`, `readline()`, `readlines()`.
- ♣ File copy: copy, verify, display contents and sizes of both files.
- ♣ Tuple of integers (max, min, count, index); set operations; Employee ID dictionary operations.

Course Module 5 Topics - Artificial Intelligence and Machine Learning (8L + 6P)

- ♣ **Artificial Intelligence:** Introduction, history and evolution of AI, applications of AI
- ♣ **Intelligent Agents:** Agents and environments, structure of agents
- ♣ **Machine Learning:** Introduction; supervised, unsupervised, and reinforcement learning
- ♣ **ML Workflow / Pipeline:** Structure of data, data pre-processing, features, feature selection, model selection, training, testing, and evaluation

Module 5 Laboratory - Working with a Dataset

- ♣ Load the Titanic dataset (`Titanic.csv`) using Pandas.
- ♣ Display the first five records, dataset information, and summary statistics.
- ♣ Select and display the input features and the target attribute.
- ♣ Split the dataset into training and testing sets using Scikit-learn.
- ♣ Display the number of training and testing samples.

At the end of this course, the student will be able to:

- 1 **DESCRIBE** modern computing approaches and their characteristics.
- 2 **DESCRIBE** fundamental Python programming constructs.
- 3 **DESCRIBE** the basics of artificial intelligence and machine learning.
- 4 **WRITE** Python programs to solve simple computational problems and perform various file operations.
- 5 **IDENTIFY** suitable Python built-in data structures and utilize them for problem solving.
- 6 **DEVELOP** Python programs utilizing NumPy, Pandas, and Scikit-learn to process given datasets.

- ❶ Kai Hwang, Geoffrey C. Fox and Jack J. Dongarra, *Distributed and Cloud Computing: From Parallel Processing to the Internet of Things*, Morgan Kaufmann, Elsevier, 2012.
- ❷ Rajkumar Buyya, Christian Vecchiola, S. Thamarai Selvi, *Mastering Cloud Computing*, McGraw Hill Education, 2013.
- ❸ Reema Thareja, *Python Programming using Problem Solving Approach*, Oxford University Press, 3rd Edition, 2024.
- ❹ Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, 4th Edition, Pearson, 2021.
- ❺ Saikat Dutt, Subramanian Chandramouli, Amit Kumar Das, *Machine Learning*, Pearson Education India, 1st Edition, 2018.

Reference Books:

- ❶ Perry Lea, *IoT and Edge Computing for Architects*, Packt Publishing, 2nd Edition, 2020.
- ❷ Eric Matthes, *Python Crash Course*, No Starch Press, 2015.
- ❸ Al Sweigart, *Automate the Boring Stuff with Python*, No Starch Press, 2019.
- ❹ Mark Fenner, *Machine Learning with Python for Everyone*, Addison-Wesley Professional, 2020.

Web Links (NPTEL):

- ♣ Python full course: <https://nptel.ac.in/courses/106106182>
- ♣ Fundamentals of AI: <https://nptel.ac.in/courses/112103280>

Tools Used in this Course

Category	Tools
Programming Language	Python 3
Numerical Computing	NumPy
Data Handling	Pandas
Machine Learning	Scikit-learn
Dataset (Lab)	Titanic dataset (<code>Titanic.csv</code>)

Thank You

❶ Scalable Computing over the Internet — why one computer is no longer enough.

❶ The Age of Internet Computing:

- ▶ Platform evolution (five generations of computers)
- ▶ High-Performance Computing (HPC) and High-Throughput Computing (HTC)
- ▶ Three new computing paradigms and computing paradigm distinctions
- ▶ Distributed system families and their design objectives

❶ Scalable Computing Trends and New Paradigms:

- ▶ Degrees of parallelism, innovative applications
- ▶ The trend toward utility computing and the hype cycle of new technologies

❶ **Scalable Computing over the Internet** — why one computer is no longer enough.

❷ **The Age of Internet Computing:**

- ▶ Platform evolution (five generations of computers)
- ▶ High-Performance Computing (HPC) and High-Throughput Computing (HTC)
- ▶ Three new computing paradigms and computing paradigm distinctions
- ▶ Distributed system families and their design objectives

❸ **Scalable Computing Trends and New Paradigms:**

- ▶ Degrees of parallelism, innovative applications
- ▶ The trend toward utility computing and the hype cycle of new technologies

- ❶ **Scalable Computing over the Internet** — why one computer is no longer enough.
- ❷ **The Age of Internet Computing:**
 - ▶ Platform evolution (five generations of computers)
 - ▶ High-Performance Computing (HPC) and High-Throughput Computing (HTC)
 - ▶ Three new computing paradigms and computing paradigm distinctions
 - ▶ Distributed system families and their design objectives
- ❸ **Scalable Computing Trends and New Paradigms:**
 - ▶ Degrees of parallelism, innovative applications
 - ▶ The trend toward utility computing and the hype cycle of new technologies

Scalable Computing over the Internet

Key Idea

Instead of using one **centralized** computer, a **parallel and distributed** computing system uses **many computers** working together over the Internet to solve large-scale problems.

- ➊ Over the past 60 years, computing has changed in its **machine architecture, operating system platform, network connectivity, and application workload**.
- ➋ As a result, computing has become **data-intensive** and **network-centric**.
- ➌ Large-scale Internet applications (search, e-mail, online banking, social media) have greatly improved the **quality of life** and information services in society.
- ➍ **Everyday example:** A single Google search is answered by thousands of servers working together, not by one computer.

Scalable Computing over the Internet

Key Idea

Instead of using one **centralized** computer, a **parallel and distributed** computing system uses **many computers** working together over the Internet to solve large-scale problems.

- 1 Over the past 60 years, computing has changed in its **machine architecture, operating system platform, network connectivity, and application workload**.
- 2 As a result, computing has become **data-intensive** and **network-centric**.
- 3 Large-scale Internet applications (search, e-mail, online banking, social media) have greatly improved the **quality of life** and information services in society.
- 4 **Everyday example:** A single Google search is answered by thousands of servers working together, not by one computer.

Scalable Computing over the Internet

Key Idea

Instead of using one **centralized** computer, a **parallel and distributed** computing system uses **many computers** working together over the Internet to solve large-scale problems.

- ➊ Over the past 60 years, computing has changed in its **machine architecture, operating system platform, network connectivity, and application workload**.
- ➋ As a result, computing has become **data-intensive** and **network-centric**.
- ➌ Large-scale Internet applications (search, e-mail, online banking, social media) have greatly improved the **quality of life** and information services in society.
- ➍ **Everyday example:** A single Google search is answered by thousands of servers working together, not by one computer.

Scalable Computing over the Internet

Key Idea

Instead of using one **centralized** computer, a **parallel and distributed** computing system uses **many computers** working together over the Internet to solve large-scale problems.

- ➊ Over the past 60 years, computing has changed in its **machine architecture, operating system platform, network connectivity, and application workload**.
- ➋ As a result, computing has become **data-intensive** and **network-centric**.
- ➌ Large-scale Internet applications (search, e-mail, online banking, social media) have greatly improved the **quality of life** and information services in society.
- ➍ **Everyday example:** A single Google search is answered by thousands of servers working together, not by one computer.

The Age of Internet Computing

- ❶ **Billions of people** use the Internet every day.
- ❷ Supercomputer sites and large **data centers** must serve huge numbers of users **at the same time** (concurrently).
- ❸ The **Linpack Benchmark**, traditionally used to rank HPC systems by raw speed, is **no longer the best measure** for such workloads.
- ❹ Cloud computing instead demands **High-Throughput Computing (HTC)** systems built with parallel and distributed technologies.
- ❺ Data centers must be upgraded with:
 - ▶ Fast servers
 - ▶ Large storage systems
 - ▶ High-bandwidth networks

The Age of Internet Computing

- ❶ **Billions of people** use the Internet every day.
- ❷ Supercomputer sites and large **data centers** must serve huge numbers of users **at the same time** (concurrently).
- ❸ The **Linpack Benchmark**, traditionally used to rank HPC systems by raw speed, is **no longer the best measure** for such workloads.
- ❹ Cloud computing instead demands **High-Throughput Computing (HTC)** systems built with parallel and distributed technologies.
- ❺ Data centers must be upgraded with:
 - ▶ Fast servers
 - ▶ Large storage systems
 - ▶ High-bandwidth networks

The Age of Internet Computing

- ❶ **Billions of people** use the Internet every day.
- ❷ Supercomputer sites and large **data centers** must serve huge numbers of users **at the same time** (concurrently).
- ❸ The **Linpack Benchmark**, traditionally used to rank HPC systems by raw speed, is **no longer the best measure** for such workloads.
- ❹ Cloud computing instead demands **High-Throughput Computing (HTC)** systems built with parallel and distributed technologies.
- ❺ Data centers must be upgraded with:
 - ▶ Fast servers
 - ▶ Large storage systems
 - ▶ High-bandwidth networks

The Age of Internet Computing

- ❶ **Billions of people** use the Internet every day.
- ❷ Supercomputer sites and large **data centers** must serve huge numbers of users **at the same time** (concurrently).
- ❸ The **Linpack Benchmark**, traditionally used to rank HPC systems by raw speed, is **no longer the best measure** for such workloads.
- ❹ Cloud computing instead demands **High-Throughput Computing (HTC)** systems built with parallel and distributed technologies.
- ❺ Data centers must be upgraded with:
 - ▶ Fast servers
 - ▶ Large storage systems
 - ▶ High-bandwidth networks

The Age of Internet Computing

- ❶ **Billions of people** use the Internet every day.
- ❷ Supercomputer sites and large **data centers** must serve huge numbers of users **at the same time** (concurrently).
- ❸ The **Linpack Benchmark**, traditionally used to rank HPC systems by raw speed, is **no longer the best measure** for such workloads.
- ❹ Cloud computing instead demands **High-Throughput Computing (HTC)** systems built with parallel and distributed technologies.
- ❺ Data centers must be upgraded with:
 - ▶ Fast servers
 - ▶ Large storage systems
 - ▶ High-bandwidth networks

The Platform Evolution: Five Generations

- ❶ Computer technology has gone through **five generations**; each lasted **10–20 years**, and successive generations **overlapped by about 10 years**.

Period	Platform	Examples	Main Users
1950–1970	Mainframes	IBM 360, CDC 6400	Large businesses, government
1960–1980	Minicomputers (lower cost)	DEC PDP 11, VAX Series	Small businesses, colleges
1970–1990	Personal computers	PCs built with VLSI microprocessors	Individuals, offices
1980–2000	Portable & pervasive devices	Laptops, wired and wireless devices	Mobile users
Since 1990	HPC and HTC systems	Clusters, grids, Internet clouds	Consumers and web-scale services

- ❷ General trend: Leverage shared web resources and massive amounts of data over the Internet.

The Platform Evolution: Five Generations

- ❶ Computer technology has gone through **five generations**; each lasted **10–20 years**, and successive generations **overlapped by about 10 years**.

Period	Platform	Examples	Main Users
1950–1970	Mainframes	IBM 360, CDC 6400	Large businesses, government
1960–1980	Minicomputers (lower cost)	DEC PDP 11, VAX Series	Small businesses, colleges
1970–1990	Personal computers	PCs built with VLSI microprocessors	Individuals, offices
1980–2000	Portable & pervasive devices	Laptops, wired and wireless devices	Mobile users
Since 1990	HPC and HTC systems	Clusters, grids, Internet clouds	Consumers and web-scale services

- ❷ **General trend:** Leverage **shared web resources** and **massive amounts of data** over the Internet.

Evolutionary Trend toward Parallel, Distributed, and Cloud Computing

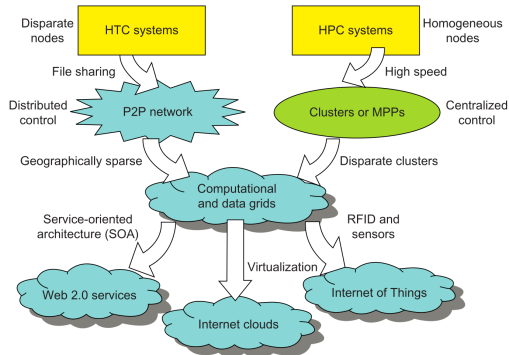


Figure: Evolutionary trend toward parallel, distributed, and cloud computing (Fig. 1.1)

- 1 The figure has **two starting points: HPC systems (right) and HTC systems (left).**
- 2 **HPC side:** Supercomputers (MPPs) are gradually replaced by clusters of cooperative computers to share resources.
- 3 **HTC side:** Peer-to-peer (P2P) networks are formed for distributed file sharing and content delivery.
- 4 Clustering and P2P technologies together lead to computational and data grids.
- 5 Grids in turn lead to Web 2.0 services, Internet clouds, and the Internet of Things.

Evolutionary Trend toward Parallel, Distributed, and Cloud Computing

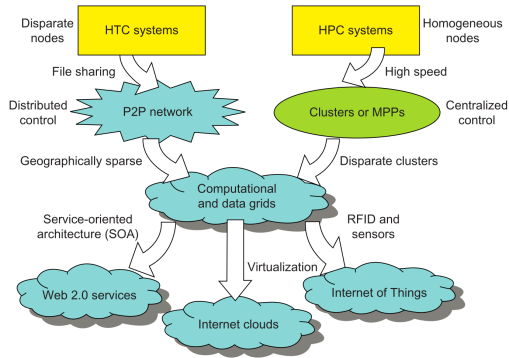


Figure: Evolutionary trend toward parallel, distributed, and cloud computing (Fig. 1.1)

- 1 The figure has **two starting points: HPC systems (right) and HTC systems (left).**
- 2 **HPC side:** Supercomputers (**MPPs**) are gradually replaced by **clusters** of cooperative computers to **share resources**.
- 3 **HTC side:** Peer-to-peer (**P2P**) networks are formed for distributed file sharing and content delivery.
- 4 Clustering and P2P technologies together lead to **computational and data grids**.
- 5 Grids in turn lead to **Web 2.0 services, Internet clouds, and the Internet of Things**.

Evolutionary Trend toward Parallel, Distributed, and Cloud Computing

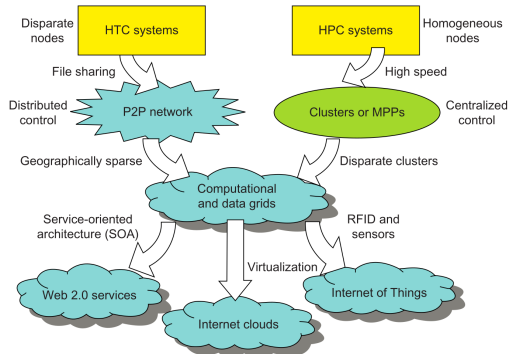


Figure: Evolutionary trend toward parallel, distributed, and cloud computing (Fig. 1.1)

- 1 The figure has **two starting points: HPC systems (right) and HTC systems (left).**
- 2 **HPC side:** Supercomputers (**MPPs**) are gradually replaced by **clusters** of cooperative computers to **share resources**.
- 3 **HTC side: Peer-to-peer (P2P)** networks are formed for distributed file sharing and content delivery.
- 4 Clustering and P2P technologies together lead to computational and data grids.
- 5 Grids in turn lead to Web 2.0 services, Internet clouds, and the Internet of Things.

Evolutionary Trend toward Parallel, Distributed, and Cloud Computing

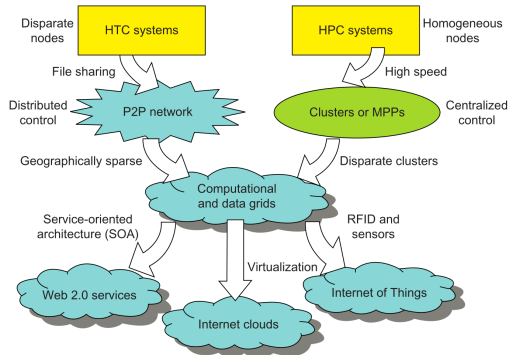


Figure: Evolutionary trend toward parallel, distributed, and cloud computing (Fig. 1.1)

- 1 The figure has **two starting points: HPC systems (right) and HTC systems (left).**
 - 2 **HPC side:** Supercomputers (**MPPs**) are gradually replaced by **clusters** of cooperative computers to **share resources**.
 - 3 **HTC side:** **Peer-to-peer (P2P)** networks are formed for distributed file sharing and content delivery.
 - 4 Clustering and P2P technologies together lead to **computational and data grids**.
- Grids in turn lead to Web 2.0 services, Internet clouds, and the Internet of Things.

Evolutionary Trend toward Parallel, Distributed, and Cloud Computing

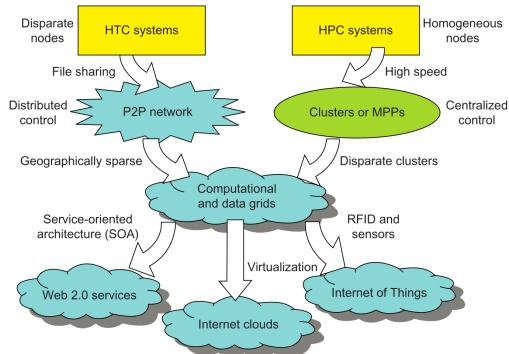


Figure: Evolutionary trend toward parallel, distributed, and cloud computing (Fig. 1.1)

- 1 The figure has **two starting points: HPC systems (right) and HTC systems (left).**
- 2 **HPC side:** Supercomputers (**MPPs**) are gradually replaced by **clusters** of cooperative computers to **share resources**.
- 3 **HTC side:** **Peer-to-peer (P2P)** networks are formed for distributed file sharing and content delivery.
- 4 Clustering and P2P technologies together lead to **computational and data grids**.
- 5 Grids in turn lead to **Web 2.0 services, Internet clouds, and the Internet of Things**.

Reading Figure 1.1: Labels on the Arrows

Feature	HPC Path (Clusters / MPPs)	HTC Path (P2P Networks)
Type of nodes	Homogeneous (same kind of computers)	Disparate (different kinds of computers)
Control	Centralized control	Distributed control
Location	Physically close (one room / building)	Geographically sparse (spread world-wide)
Purpose	High speed	File sharing

- ❶ **Disparate clusters and geographically sparse P2P networks merge into grids.**
- ❷ From grids: SOA → Web 2.0 services; Virtualization → Internet clouds; RFID and sensors → Internet of Things.
- ❸ A **cluster** is a collection of homogeneous compute nodes physically connected close to one another (detailed later in this module).

Reading Figure 1.1: Labels on the Arrows

Feature	HPC Path (Clusters / MPPs)	HTC Path (P2P Networks)
Type of nodes	Homogeneous (same kind of computers)	Disparate (different kinds of computers)
Control	Centralized control	Distributed control
Location	Physically close (one room / building)	Geographically sparse (spread world-wide)
Purpose	High speed	File sharing

- ➊ **Disparate clusters** and **geographically sparse** P2P networks merge into **grids**.
- ➋ From grids: **SOA** → Web 2.0 services; **Virtualization** → Internet clouds; **RFID and sensors** → Internet of Things.
- ➌ A **cluster** is a collection of homogeneous compute nodes physically connected close to one another (detailed later in this module).

Reading Figure 1.1: Labels on the Arrows

Feature	HPC Path (Clusters / MPPs)	HTC Path (P2P Networks)
Type of nodes	Homogeneous (same kind of computers)	Disparate (different kinds of computers)
Control	Centralized control	Distributed control
Location	Physically close (one room / building)	Geographically sparse (spread world-wide)
Purpose	High speed	File sharing

- ❶ **Disparate clusters** and **geographically sparse** P2P networks merge into **grids**.
- ❷ From grids: **SOA** → Web 2.0 services; **Virtualization** → Internet clouds; **RFID and sensors** → Internet of Things.
- ❸ A **cluster** is a collection of homogeneous compute nodes physically connected close to one another (detailed later in this module).

High-Performance Computing (HPC)

HPC

Computing systems that emphasize **raw speed** performance — how fast a single large problem can be solved.

- ① HPC speed has grown from **Gflops** (early 1990s) to **Pflops** (2010).
 - ▶ flops = floating-point operations per second; Giga = 10^9 , Peta = 10^{15} .
- ② Growth was driven by scientific, engineering, and manufacturing communities.
- ③ The **Top 500** list ranks the world's most powerful computers by floating-point speed in the **Linpack benchmark**.
- ④ However, supercomputer users are **less than 10%** of all computer users.
- ⑤ **Example:** Weather forecasting, earthquake simulation, genome analysis.

High-Performance Computing (HPC)

HPC

Computing systems that emphasize **raw speed** performance — how fast a single large problem can be solved.

- ① HPC speed has grown from **Gflops** (early 1990s) to **Pflops** (2010).
 - ▶ flops = floating-point operations per second; Giga = 10^9 , Peta = 10^{15} .
- ② Growth was driven by **scientific, engineering, and manufacturing** communities.
- ③ The **Top 500** list ranks the world's most powerful computers by floating-point speed in the **Linpack benchmark**.
- ④ However, supercomputer users are **less than 10%** of all computer users.
- ⑤ **Example:** Weather forecasting, earthquake simulation, genome analysis.

High-Performance Computing (HPC)

HPC

Computing systems that emphasize **raw speed** performance — how fast a single large problem can be solved.

- ① HPC speed has grown from **Gflops** (early 1990s) to **Pflops** (2010).
 - ▶ flops = floating-point operations per second; Giga = 10^9 , Peta = 10^{15} .
- ② Growth was driven by **scientific, engineering, and manufacturing** communities.
- ③ The **Top 500** list ranks the world's most powerful computers by floating-point speed in the **Linpack benchmark**.
- ④ However, supercomputer users are **less than 10%** of all computer users.
- ④ Example: Weather forecasting, earthquake simulation, genome analysis.

High-Performance Computing (HPC)

HPC

Computing systems that emphasize **raw speed** performance — how fast a single large problem can be solved.

- ① HPC speed has grown from **Gflops** (early 1990s) to **Pflops** (2010).
 - ▶ flops = floating-point operations per second; Giga = 10^9 , Peta = 10^{15} .
 - ② Growth was driven by **scientific, engineering, and manufacturing** communities.
 - ③ The **Top 500** list ranks the world's most powerful computers by floating-point speed in the **Linpack benchmark**.
 - ④ However, supercomputer users are **less than 10%** of all computer users.
- Example: Weather forecasting, earthquake simulation, genome analysis.

High-Performance Computing (HPC)

HPC

Computing systems that emphasize **raw speed** performance — how fast a single large problem can be solved.

- ① HPC speed has grown from **Gflops** (early 1990s) to **Pflops** (2010).
 - ▶ flops = floating-point operations per second; Giga = 10^9 , Peta = 10^{15} .
- ② Growth was driven by **scientific, engineering, and manufacturing** communities.
- ③ The **Top 500** list ranks the world's most powerful computers by floating-point speed in the **Linpac benchmark**.
- ④ However, supercomputer users are **less than 10%** of all computer users.
- ⑤ **Example:** Weather forecasting, earthquake simulation, genome analysis.

High-Throughput Computing (HTC)

HTC

Computing systems that emphasize **high-flux computing** — the **number of tasks completed per unit of time**.

- ➊ Market-oriented high-end computing is shifting from an **HPC paradigm** to an **HTC paradigm**.
- ➋ Main applications: **Internet searches** and **web services** used by millions of users simultaneously.
- ➌ HTC must improve batch processing speed **and** address:
 - ▶ Cost
 - ▶ Energy savings
 - ▶ Security
 - ▶ Reliability
- ➍ **Example:** Millions of students checking exam results on a university website at the same time.

High-Throughput Computing (HTC)

HTC

Computing systems that emphasize **high-flux computing** — the **number of tasks completed per unit of time**.

- ❶ Market-oriented high-end computing is shifting from an **HPC paradigm** to an **HTC paradigm**.
- ❷ Main applications: **Internet searches** and **web services** used by millions of users simultaneously.
- ❸ HTC must improve batch processing speed and address:
 - ▶ Cost
 - ▶ Energy savings
 - ▶ Security
 - ▶ Reliability
- ❹ **Example:** Millions of students checking exam results on a university website at the same time.

High-Throughput Computing (HTC)

HTC

Computing systems that emphasize **high-flux computing** — the **number of tasks completed per unit of time**.

- ❶ Market-oriented high-end computing is shifting from an **HPC paradigm** to an **HTC paradigm**.
- ❷ Main applications: **Internet searches** and **web services** used by millions of users simultaneously.
- ❸ HTC must improve batch processing speed **and** address:
 - ▶ Cost
 - ▶ Energy savings
 - ▶ Security
 - ▶ Reliability

❹ **Example:** Millions of students checking exam results on a university website at the same time.

High-Throughput Computing (HTC)

HTC

Computing systems that emphasize **high-flux computing** — the **number of tasks completed per unit of time**.

- ❶ Market-oriented high-end computing is shifting from an **HPC paradigm** to an **HTC paradigm**.
- ❷ Main applications: **Internet searches** and **web services** used by millions of users simultaneously.
- ❸ HTC must improve batch processing speed **and** address:
 - ▶ Cost
 - ▶ Energy savings
 - ▶ Security
 - ▶ Reliability
- ❹ **Example:** Millions of students checking exam results on a university website at the same time.

HPC versus HTC: A Comparison

Aspect	High-Performance Computing	High-Throughput Computing
Goal	Solve one big problem fast	Complete many tasks per unit time
Measure	Speed (flops, Linpack benchmark)	Throughput (tasks completed per unit time)
Typical systems	Supercomputers, MPPs, clusters	P2P networks, clouds, web service platforms
Typical users	Scientists and engineers (<10% of users)	General Internet users (millions)
Applications	Simulations, weather forecasting	Web search, e-commerce, social networks

- ❶ **Analogy:** HPC is a **racing car** (one passenger, very fast); HTC is a **metro train** (thousands of passengers per hour).

Three New Computing Paradigms

❶ **Service-Oriented Architecture (SOA) → Web 2.0 services:**

- ▶ Software is offered as **services** over the web (e.g., online maps, payment services).

❷ **Virtualization → Internet clouds:**

- ▶ Advances in virtualization made it possible to grow **Internet clouds** as a new paradigm.

❸ **RFID, GPS, and sensor technologies → Internet of Things (IoT):**

- ▶ Maturity of radio-frequency identification (RFID), Global Positioning System (GPS), and sensors triggered the development of the IoT.

❹ **The differences among clusters, grids, P2P systems, and clouds may blur in the future.**

Three New Computing Paradigms

❶ **Service-Oriented Architecture (SOA) → Web 2.0 services:**

- ▶ Software is offered as **services** over the web (e.g., online maps, payment services).

❷ **Virtualization → Internet clouds:**

- ▶ Advances in virtualization made it possible to grow **Internet clouds** as a new paradigm.

❸ **RFID, GPS, and sensor technologies → Internet of Things (IoT):**

- ▶ Maturity of radio-frequency identification (RFID), Global Positioning System (GPS), and sensors triggered the development of the IoT.

❹ **The differences among clusters, grids, P2P systems, and clouds may blur in the future.**

Three New Computing Paradigms

❶ **Service-Oriented Architecture (SOA) → Web 2.0 services:**

- ▶ Software is offered as **services** over the web (e.g., online maps, payment services).

❷ **Virtualization → Internet clouds:**

- ▶ Advances in virtualization made it possible to grow **Internet clouds** as a new paradigm.

❸ **RFID, GPS, and sensor technologies → Internet of Things (IoT):**

- ▶ Maturity of radio-frequency identification (RFID), Global Positioning System (GPS), and sensors triggered the development of the IoT.

❹ The differences among clusters, grids, P2P systems, and clouds may blur in the future.

Three New Computing Paradigms

- ❶ **Service-Oriented Architecture (SOA) → Web 2.0 services:**
 - ▶ Software is offered as **services** over the web (e.g., online maps, payment services).
- ❷ **Virtualization → Internet clouds:**
 - ▶ Advances in virtualization made it possible to grow **Internet clouds** as a new paradigm.
- ❸ **RFID, GPS, and sensor technologies → Internet of Things (IoT):**
 - ▶ Maturity of radio-frequency identification (RFID), Global Positioning System (GPS), and sensors triggered the development of the IoT.
- ❹ The differences among clusters, grids, P2P systems, and clouds **may blur** in the future.

How the Idea of a “Computer” Has Changed

Year	Person	Idea
1969	Leonard Kleinrock (UCLA)	Computer networks would grow into computer utilities , serving homes and offices like electricity and telephone.
1984	John Gage (Sun Microsystems)	“The network is the computer.”
2008	David Patterson (UC Berkeley)	“The data center is the computer.”
Recent	Rajkumar Buyya (Univ. of Melbourne)	“The cloud is the computer.”

- 1 The “computer” has moved from a **single machine** → a **network** → a **data center** → the **cloud**.
- 2 Building software used as a **service by millions** is very different from distributing software to run on individual PCs.

How the Idea of a “Computer” Has Changed

Year	Person	Idea
1969	Leonard Kleinrock (UCLA)	Computer networks would grow into computer utilities , serving homes and offices like electricity and telephone.
1984	John Gage (Sun Microsystems)	“The network is the computer.”
2008	David Patterson (UC Berkeley)	“The data center is the computer.”
Recent	Rajkumar Buyya (Univ. of Melbourne)	“The cloud is the computer.”

- 1 The “computer” has moved from a **single machine** → a **network** → a **data center** → the **cloud**.
- 2 Building software used as a **service by millions** is very different from distributing software to run on individual PCs.

Computing Paradigm Distinctions (1/2)

① Centralized computing:

- ▶ All resources (processors, memory, storage) are in **one physical system**.
- ▶ Resources are fully shared and **tightly coupled** within one integrated OS.
- ▶ Many data centers and supercomputers are centralized systems.

② Parallel computing:

- ▶ Processors are either **tightly coupled** (centralized shared memory) or **loosely coupled** (distributed memory).
- ▶ Processors communicate through **shared memory** or **message passing**.
- ▶ The computer is called a **parallel computer**; its programs are **parallel programs**; writing them is **parallel programming**.

Computing Paradigm Distinctions (1/2)

① Centralized computing:

- ▶ All resources (processors, memory, storage) are in **one physical system**.
- ▶ Resources are fully shared and **tightly coupled** within one integrated OS.
- ▶ Many data centers and supercomputers are centralized systems.

② Parallel computing:

- ▶ Processors are either **tightly coupled** (centralized shared memory) or **loosely coupled** (distributed memory).
- ▶ Processors communicate through **shared memory** or **message passing**.
- ▶ The computer is called a **parallel computer**; its programs are **parallel programs**; writing them is **parallel programming**.

① Distributed computing:

- ▶ A distributed system has **multiple autonomous computers**, each with its **own private memory**, connected by a network.
- ▶ Information is exchanged through **message passing**.
- ▶ Programs are **distributed programs**; writing them is **distributed programming**.

② Cloud computing:

- ▶ An Internet cloud can be either **centralized or distributed**.
- ▶ It applies parallel or distributed computing, or both.
- ▶ Built with **physical or virtualized** resources over large data centers.
- ▶ Some consider it a form of **utility computing or service computing**.

Computing Paradigm Distinctions (2/2)

① Distributed computing:

- ▶ A distributed system has **multiple autonomous computers**, each with its **own private memory**, connected by a network.
- ▶ Information is exchanged through **message passing**.
- ▶ Programs are **distributed programs**; writing them is **distributed programming**.

② Cloud computing:

- ▶ An Internet cloud can be either **centralized** or **distributed**.
- ▶ It applies parallel or distributed computing, or both.
- ▶ Built with **physical or virtualized** resources over large data centers.
- ▶ Some consider it a form of **utility computing** or **service computing**.

Computing Paradigms at a Glance

Paradigm	Memory	Communication	Everyday Analogy
Centralized	One system, fully shared	Within one OS	One chef cooking in one kitchen
Parallel	Shared or distributed	Shared memory / message passing	Many chefs in one kitchen cooking one meal
Distributed	Private memory per computer	Message passing over a network	Branches of a restaurant chain coordinating by phone
Cloud	Centralized or distributed	Over the Internet	Ordering food online; you never see the kitchen

❶ **Distributed computing** is the **opposite** of centralized computing.

❷ Parallel computing overlaps with distributed computing; cloud computing overlaps with all three.

Computing Paradigms at a Glance

Paradigm	Memory	Communication	Everyday Analogy
Centralized	One system, fully shared	Within one OS	One chef cooking in one kitchen
Parallel	Shared or distributed	Shared memory / message passing	Many chefs in one kitchen cooking one meal
Distributed	Private memory per computer	Message passing over a network	Branches of a restaurant chain coordinating by phone
Cloud	Centralized or distributed	Over the Internet	Ordering food online; you never see the kitchen

- 1 **Distributed computing** is the **opposite** of centralized computing.
- 2 Parallel computing **overlaps** with distributed computing; cloud computing overlaps with all three.

- ❶ **Concurrent computing / programming:** Usually refers to the **union** of parallel and distributed computing.
- ❷ **Ubiquitous computing:** Computing with pervasive devices at **any place and any time**, using wired or wireless communication.
- ❸ **Internet of Things (IoT):** A networked connection of **everyday objects** — computers, sensors, humans, etc. — supported by Internet clouds.
- ❹ **Internet computing:** The broadest term, covering **all computing paradigms** over the Internet.
- ❺ **Example of IoT:** A smart watch sending your step count to a phone app, which stores it in the cloud.

- ❶ **Concurrent computing / programming:** Usually refers to the **union** of parallel and distributed computing.
- ❷ **Ubiquitous computing:** Computing with pervasive devices at **any place and any time**, using wired or wireless communication.
- ❸ **Internet of Things (IoT):** A networked connection of **everyday objects** — computers, sensors, humans, etc. — supported by Internet clouds.
- ❹ **Internet computing:** The broadest term, covering **all computing paradigms** over the Internet.
- ❺ **Example of IoT:** A smart watch sending your step count to a phone app, which stores it in the cloud.

- ❶ **Concurrent computing / programming:** Usually refers to the **union** of parallel and distributed computing.
- ❷ **Ubiquitous computing:** Computing with pervasive devices at **any place and any time**, using wired or wireless communication.
- ❸ **Internet of Things (IoT):** A networked connection of **everyday objects** — computers, sensors, humans, etc. — supported by Internet clouds.
- ❹ **Internet computing:** The broadest term, covering all computing paradigms over the Internet.
- ❺ **Example of IoT:** A smart watch sending your step count to a phone app, which stores it in the cloud.

- ❶ **Concurrent computing / programming:** Usually refers to the **union** of parallel and distributed computing.
- ❷ **Ubiquitous computing:** Computing with pervasive devices at **any place and any time**, using wired or wireless communication.
- ❸ **Internet of Things (IoT):** A networked connection of **everyday objects** — computers, sensors, humans, etc. — supported by Internet clouds.
- ❹ **Internet computing:** The broadest term, covering **all computing paradigms** over the Internet.
- ❺ **Example of IoT:** A smart watch sending your step count to a phone app, which stores it in the cloud.

- ❶ **Concurrent computing / programming:** Usually refers to the **union** of parallel and distributed computing.
- ❷ **Ubiquitous computing:** Computing with pervasive devices at **any place and any time**, using wired or wireless communication.
- ❸ **Internet of Things (IoT):** A networked connection of **everyday objects** — computers, sensors, humans, etc. — supported by Internet clouds.
- ❹ **Internet computing:** The broadest term, covering **all computing paradigms** over the Internet.
- ❺ **Example of IoT:** A smart watch sending your step count to a phone app, which stores it in the cloud.

- ❶ Since the mid-1990s, P2P networks and networks of clusters were consolidated into national projects called **computational grids** or **data grids**.
- ❷ Internet clouds result from moving desktop computing to service-oriented computing using server clusters and huge databases at data centers.
- ❸ Grids and clouds place great emphasis on **resource sharing** in hardware, software, and data sets.
- ❹ Scale of these systems:
 - ▶ Tianhe-1A cluster (China, Oct. 2010): 86,016 CPU cores and 3,211,264 GPU cores.
 - ▶ The largest computational grid connects up to **hundreds of server clusters**.
 - ▶ A typical P2P network may involve **millions of client machines**.
 - ▶ Experimental cloud clusters have been built with **thousands of processing nodes**.

- ❶ Since the mid-1990s, P2P networks and networks of clusters were consolidated into national projects called **computational grids** or **data grids**.
- ❷ **Internet clouds** result from moving desktop computing to **service-oriented computing** using server clusters and huge databases at data centers.
- ❸ Grids and clouds place great emphasis on **resource sharing** in hardware, software, and data sets.
- ❹ **Scale of these systems:**
 - ▶ Tianhe-1A cluster (China, Oct. 2010): 86,016 CPU cores and 3,211,264 GPU cores.
 - ▶ The largest computational grid connects up to **hundreds of server clusters**.
 - ▶ A typical P2P network may involve **millions of client machines**.
 - ▶ Experimental cloud clusters have been built with **thousands of processing nodes**.

- ① Since the mid-1990s, P2P networks and networks of clusters were consolidated into national projects called **computational grids** or **data grids**.
- ② **Internet clouds** result from moving desktop computing to **service-oriented computing** using server clusters and huge databases at data centers.
- ③ Grids and clouds place great emphasis on **resource sharing** in hardware, software, and data sets.

④ Scale of these systems:

- ▶ Tianhe-1A cluster (China, Oct. 2010): 86,016 CPU cores and 3,211,264 GPU cores.
- ▶ The largest computational grid connects up to **hundreds of server clusters**.
- ▶ A typical P2P network may involve **millions of client machines**.
- ▶ Experimental cloud clusters have been built with **thousands of processing nodes**.

- ❶ Since the mid-1990s, P2P networks and networks of clusters were consolidated into national projects called **computational grids** or **data grids**.
- ❷ **Internet clouds** result from moving desktop computing to **service-oriented computing** using server clusters and huge databases at data centers.
- ❸ Grids and clouds place great emphasis on **resource sharing** in hardware, software, and data sets.
- ❹ **Scale of these systems:**
 - ▶ **Tianhe-1A** cluster (China, Oct. 2010): 86,016 CPU cores and 3,211,264 GPU cores.
 - ▶ The largest computational grid connects up to **hundreds of server clusters**.
 - ▶ A typical P2P network may involve **millions of client machines**.
 - ▶ Experimental cloud clusters have been built with **thousands of processing nodes**.

Design Objectives for Future HPC and HTC Systems

- ❶ Future systems will need **multicore** / **many-core** processors handling many threads per core.
- ❷ System efficiency depends on speed, programming, and energy (throughput per watt).

Objective	What It Measures
Efficiency	Utilization of resources; for HPC via massive parallelism, for HTC via job throughput, data access, storage, and power efficiency.
Dependability	Reliability and self-management from chip to application level; service with QoS assurance even under failures.
Adaptation in the programming model	Ability to support billions of job requests over massive data sets and virtualized cloud resources.
Flexibility in application deployment	Ability to run well in both HPC (science and engineering) and HTC (business) applications.

Design Objectives for Future HPC and HTC Systems

- 1 Future systems will need **multicore** / **many-core** processors handling many threads per core.
- 2 System efficiency depends on **speed**, **programming**, and **energy** (throughput per watt).

Objective	What It Measures
Efficiency	Utilization of resources; for HPC via massive parallelism, for HTC via job throughput, data access, storage, and power efficiency.
Dependability	Reliability and self-management from chip to application level; service with QoS assurance even under failures.
Adaptation in the programming model	Ability to support billions of job requests over massive data sets and virtualized cloud resources.
Flexibility in application deployment	Ability to run well in both HPC (science and engineering) and HTC (business) applications.

Scalable Computing Trends

- ➊ Designers try to **predict** the capabilities of future systems from technology trends (e.g., Jim Gray's paper *Rules of Thumb in Data Engineering*).
- ➋ **Moore's law:** Processor speed doubles every 18 months.
 - ▶ Valid for the last 30 years; whether it continues is uncertain.
- ➌ **Gilder's law:** Network bandwidth has doubled each year.
- ➍ The excellent **price/performance ratio** of commodity hardware (desktops, notebooks, tablets) has driven the use of commodity technologies in **large-scale computing**.
- ➎ Distributed systems emphasize both **resource distribution** and **concurrency**, i.e., a high degree of **parallelism (DoP)**.

Scalable Computing Trends

- 1 Designers try to **predict** the capabilities of future systems from technology trends (e.g., Jim Gray's paper *Rules of Thumb in Data Engineering*).
- 2 **Moore's law:** Processor speed **doubles every 18 months**.
 - ▶ Valid for the last 30 years; whether it continues is uncertain.
- 3 **Gilder's law:** Network bandwidth has **doubled each year**.
- 4 The excellent **price/performance ratio** of commodity hardware (desktops, notebooks, tablets) has driven the use of commodity technologies in **large-scale computing**.
- 5 Distributed systems emphasize both **resource distribution** and **concurrency**, i.e., a high degree of **parallelism (DoP)**.

Scalable Computing Trends

- 1 Designers try to **predict** the capabilities of future systems from technology trends (e.g., Jim Gray's paper *Rules of Thumb in Data Engineering*).
- 2 **Moore's law:** Processor speed **doubles every 18 months**.
 - ▶ Valid for the last 30 years; whether it continues is uncertain.
- 3 **Gilder's law:** Network bandwidth has **doubled each year**.
- 4 The excellent **price/performance ratio** of commodity hardware (desktops, notebooks, tablets) has driven the use of commodity technologies in **large-scale computing**.
- 5 Distributed systems emphasize both **resource distribution** and **concurrency**, i.e., a high degree of **parallelism (DoP)**.

Scalable Computing Trends

- ① Designers try to **predict** the capabilities of future systems from technology trends (e.g., Jim Gray's paper *Rules of Thumb in Data Engineering*).
- ② **Moore's law:** Processor speed **doubles every 18 months**.
 - ▶ Valid for the last 30 years; whether it continues is uncertain.
- ③ **Gilder's law:** Network bandwidth has **doubled each year**.
- ④ The excellent **price/performance ratio** of commodity hardware (desktops, notebooks, tablets) has driven the use of commodity technologies in **large-scale computing**.
- ⑤ Distributed systems emphasize both **resource distribution and concurrency**, i.e., a high degree of parallelism (DoP).

Scalable Computing Trends

- ❶ Designers try to **predict** the capabilities of future systems from technology trends (e.g., Jim Gray's paper *Rules of Thumb in Data Engineering*).
- ❷ **Moore's law:** Processor speed **doubles every 18 months**.
 - ▶ Valid for the last 30 years; whether it continues is uncertain.
- ❸ **Gilder's law:** Network bandwidth has **doubled each year**.
- ❹ The excellent **price/performance ratio** of commodity hardware (desktops, notebooks, tablets) has driven the use of commodity technologies in **large-scale computing**.
- ❺ Distributed systems emphasize both **resource distribution** and **concurrency**, i.e., a high **degree of parallelism (DoP)**.

Degrees of Parallelism

Level	Description
Bit-Level Parallelism (BLP)	Converts bit-serial processing to word-level; 4-bit → 8, 16, 32, 64-bit CPUs.
Instruction-Level Parallelism (ILP)	Processor executes multiple instructions at once ; via pipelining, superscalar, VLIW, multithreading.
Data-Level Parallelism (DLP)	One instruction on many data items ; via SIMD and vector machines.
Task-Level Parallelism (TLP)	Different tasks on different cores of multicore processors (CMPs).
Job-Level Parallelism (JLP)	Whole jobs on different computers in distributed processing.

- 1 BLP, ILP, and DLP are well supported by hardware and compilers.
- 2 TLP is less successful because programming multicore processors efficiently is difficult.
- 3 Coarse-grain parallelism (JLP) is built on top of fine-grain parallelism (BLP, ILP).

Degrees of Parallelism

Level	Description
Bit-Level Parallelism (BLP)	Converts bit-serial processing to word-level; 4-bit → 8, 16, 32, 64-bit CPUs.
Instruction-Level Parallelism (ILP)	Processor executes multiple instructions at once ; via pipelining, superscalar, VLIW, multithreading.
Data-Level Parallelism (DLP)	One instruction on many data items ; via SIMD and vector machines.
Task-Level Parallelism (TLP)	Different tasks on different cores of multicore processors (CMPs).
Job-Level Parallelism (JLP)	Whole jobs on different computers in distributed processing.

- 1 BLP, ILP, and DLP are well supported by hardware and compilers.
- 2 TLP is **less successful** because programming multicore processors efficiently is difficult.
- Coarse-grain parallelism (JLP) is built on top of fine-grain parallelism (BLP, ILP).

Degrees of Parallelism

Level	Description
Bit-Level Parallelism (BLP)	Converts bit-serial processing to word-level; 4-bit → 8, 16, 32, 64-bit CPUs.
Instruction-Level Parallelism (ILP)	Processor executes multiple instructions at once ; via pipelining, superscalar, VLIW, multithreading.
Data-Level Parallelism (DLP)	One instruction on many data items ; via SIMD and vector machines.
Task-Level Parallelism (TLP)	Different tasks on different cores of multicore processors (CMPs).
Job-Level Parallelism (JLP)	Whole jobs on different computers in distributed processing.

- 1 BLP, ILP, and DLP are well supported by hardware and compilers.
- 2 TLP is **less successful** because programming multicore processors efficiently is difficult.
- 3 **Coarse-grain** parallelism (JLP) is built on top of **fine-grain** parallelism (BLP, ILP).

Degrees of Parallelism: An Everyday View

- ❶ **BLP:** Adding two numbers 64 bits at a time instead of one bit at a time.
- ❷ **ILP:** An assembly line — one instruction is fetched while another is being executed.
- ❸ **DLP:** Increasing the brightness of every pixel of a photo with one instruction.
- ❹ **TLP:** Your phone plays music on one core while downloading a file on another.
- ❺ **JLP:** A cloud runs thousands of users' jobs on thousands of separate servers.
- ❻ Granularity **increases** as we move from parallel processing (inside one computer) to distributed processing (across many computers).

Degrees of Parallelism: An Everyday View

- ❶ **BLP:** Adding two numbers 64 bits at a time instead of one bit at a time.
- ❷ **ILP:** An assembly line — one instruction is fetched while another is being executed.
- ❸ **DLP:** Increasing the brightness of every pixel of a photo with one instruction.
- ❹ **TLP:** Your phone plays music on one core while downloading a file on another.
- ❺ **JLP:** A cloud runs thousands of users' jobs on thousands of separate servers.
- ❻ Granularity **increases** as we move from parallel processing (inside one computer) to distributed processing (across many computers).

Degrees of Parallelism: An Everyday View

- ❶ **BLP:** Adding two numbers 64 bits at a time instead of one bit at a time.
- ❷ **ILP:** An assembly line — one instruction is fetched while another is being executed.
- ❸ **DLP:** Increasing the brightness of **every pixel** of a photo with one instruction.
- ❹ **TLP:** Your phone plays music on one core while downloading a file on another.
- ❺ **JLP:** A cloud runs thousands of users' jobs on thousands of separate servers.
- ❻ Granularity **increases** as we move from parallel processing (inside one computer) to distributed processing (across many computers).

Degrees of Parallelism: An Everyday View

- ❶ **BLP:** Adding two numbers 64 bits at a time instead of one bit at a time.
- ❷ **ILP:** An assembly line — one instruction is fetched while another is being executed.
- ❸ **DLP:** Increasing the brightness of **every pixel** of a photo with one instruction.
- ❹ **TLP:** Your phone plays music on one core while downloading a file on another.
- ❺ **JLP:** A cloud runs thousands of users' jobs on thousands of separate servers.
- ❻ Granularity **increases** as we move from parallel processing (inside one computer) to distributed processing (across many computers).

Degrees of Parallelism: An Everyday View

- ❶ **BLP:** Adding two numbers 64 bits at a time instead of one bit at a time.
 - ❷ **ILP:** An assembly line — one instruction is fetched while another is being executed.
 - ❸ **DLP:** Increasing the brightness of **every pixel** of a photo with one instruction.
 - ❹ **TLP:** Your phone plays music on one core while downloading a file on another.
 - ❺ **JLP:** A cloud runs thousands of users' jobs on thousands of separate servers.
- ❻ Granularity **increases** as we move from parallel processing (inside one computer) to distributed processing (across many computers).

Degrees of Parallelism: An Everyday View

- ❶ **BLP:** Adding two numbers 64 bits at a time instead of one bit at a time.
- ❷ **ILP:** An assembly line — one instruction is fetched while another is being executed.
- ❸ **DLP:** Increasing the brightness of **every pixel** of a photo with one instruction.
- ❹ **TLP:** Your phone plays music on one core while downloading a file on another.
- ❺ **JLP:** A cloud runs thousands of users' jobs on thousands of separate servers.
- ❻ Granularity **increases** as we move from parallel processing (inside one computer) to distributed processing (across many computers).

Domain	Specific Applications
Science and engineering	Scientific simulations, genomic analysis; earthquake prediction, global warming, weather forecasting
Business, education, services industry, and health care	Telecommunication, content delivery, e-commerce; banking, stock exchanges, transaction processing; air traffic control, electric power grids, distance education; health care, hospital automation, telemedicine
Internet and web services, and government	Internet search, data centers, decision-making systems; traffic monitoring, worm containment, cyber security; digital government, online tax return processing, social networking
Mission-critical applications	Military command and control, intelligent systems, crisis management

Table: Applications of high-performance and high-throughput systems (Table 1.1)

Requirements of Innovative Applications

- ❶ HPC and HTC systems need **transparency**: the user should not have to worry about:
 - ▶ Data access and resource allocation
 - ▶ Process location and concurrency in execution
 - ▶ Job replication and failure recovery
- ❷ Almost all applications demand **computing economics**, **web-scale data collection**, **system reliability**, and **scalable performance**.
- ❸ **Banking example**:
 - ▶ Distributed transaction processing is common in banking and finance.
 - ▶ Transactions form about 90% of the market for reliable banking systems.
 - ▶ Keeping **replicated transaction records consistent** is crucial in real-time banking.
- ❹ **Challenges**: lack of software support, **network saturation**, and **security threats**.

Requirements of Innovative Applications

- ❶ HPC and HTC systems need **transparency**: the user should not have to worry about:
 - ▶ Data access and resource allocation
 - ▶ Process location and concurrency in execution
 - ▶ Job replication and failure recovery
- ❷ Almost all applications demand **computing economics, web-scale data collection, system reliability, and scalable performance**.
- ❸ **Banking example:**
 - ▶ Distributed transaction processing is common in banking and finance.
 - ▶ Transactions form about 90% of the market for reliable banking systems.
 - ▶ Keeping replicated transaction records consistent is crucial in real-time banking.
- ❹ **Challenges: lack of software support, network saturation, and security threats.**

Requirements of Innovative Applications

- ❶ HPC and HTC systems need **transparency**: the user should not have to worry about:
 - ▶ Data access and resource allocation
 - ▶ Process location and concurrency in execution
 - ▶ Job replication and failure recovery
- ❷ Almost all applications demand **computing economics, web-scale data collection, system reliability, and scalable performance**.
- ❸ **Banking example**:
 - ▶ Distributed transaction processing is common in banking and finance.
 - ▶ Transactions form about **90%** of the market for reliable banking systems.
 - ▶ Keeping **replicated transaction records consistent** is crucial in real-time banking.

❹ Challenges: lack of software support, network saturation, and security threats.

Requirements of Innovative Applications

- ❶ HPC and HTC systems need **transparency**: the user should not have to worry about:
 - ▶ Data access and resource allocation
 - ▶ Process location and concurrency in execution
 - ▶ Job replication and failure recovery
- ❷ Almost all applications demand **computing economics, web-scale data collection, system reliability, and scalable performance**.
- ❸ **Banking example**:
 - ▶ Distributed transaction processing is common in banking and finance.
 - ▶ Transactions form about **90%** of the market for reliable banking systems.
 - ▶ Keeping **replicated transaction records consistent** is crucial in real-time banking.
- ❹ Challenges: lack of software support, **network saturation**, and **security threats**.

The Trend toward Utility Computing

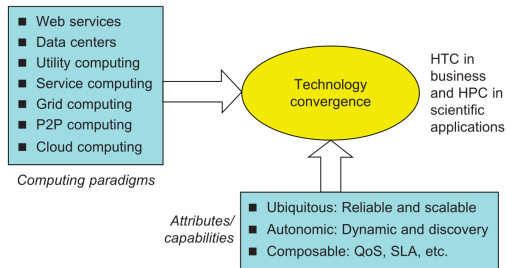


Figure: The vision of computer utilities in modern distributed computing systems (Fig. 1.2)

❶ **Left box:** Computing paradigms — web services, data centers, utility, service, grid, P2P, and cloud computing.

❷ **Bottom box:** Common attributes — ubiquitous, autonomic, and composable.

❸ Both feed into technology convergence.

❹ The result: HTC in business and HPC in scientific applications.

The Trend toward Utility Computing

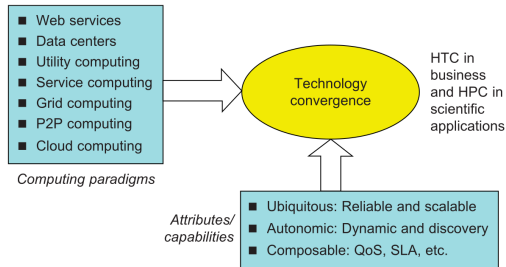


Figure: The vision of computer utilities in modern distributed computing systems (Fig. 1.2)

- 1 **Left box:** Computing paradigms — web services, data centers, utility, service, grid, P2P, and cloud computing.
 - 2 **Bottom box:** Common attributes — **ubiquitous, autonomic, and composable.**
- Both feed into technology convergence.
- The result: HTC in business and HPC in scientific applications.

The Trend toward Utility Computing

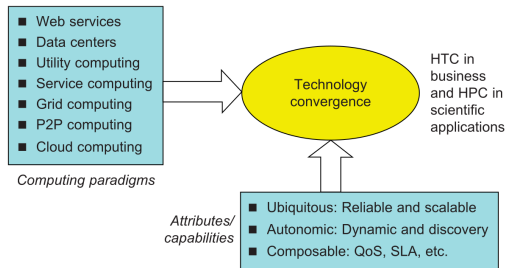


Figure: The vision of computer utilities in modern distributed computing systems (Fig. 1.2)

- 1 **Left box:** Computing paradigms — web services, data centers, utility, service, grid, P2P, and cloud computing.
- 2 **Bottom box:** Common attributes — **ubiquitous, autonomic, and composable.**
- 3 Both feed into **technology convergence.**

4 The result: **HTC in business and HPC in scientific applications.**

The Trend toward Utility Computing

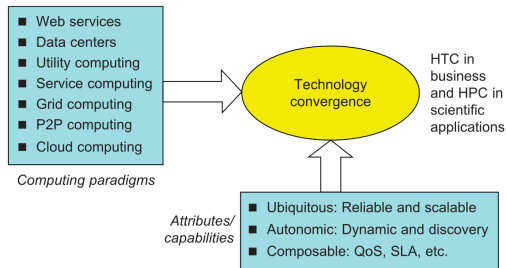


Figure: The vision of computer utilities in modern distributed computing systems (Fig. 1.2)

- 1 **Left box:** Computing paradigms — web services, data centers, utility, service, grid, P2P, and cloud computing.
- 2 **Bottom box:** Common attributes — **ubiquitous, autonomic, and composable.**
- 3 Both feed into **technology convergence.**
- 4 The result: **HTC in business** and **HPC in scientific** applications.

Common Characteristics of These Paradigms

- ❶ **Ubiquitous:** Present everywhere in daily life; **reliability** and **scalability** are the two major design objectives.
- ❷ **Autonomic:** Can self-organize to support **dynamic discovery** of resources.
- ❸ **Composable:** Can be combined with guaranteed **QoS** (Quality of Service) and **SLAs** (Service-Level Agreements).
- ❹ Together, these paradigms and attributes realize the **computer utility vision**.

Common Characteristics of These Paradigms

- ❶ **Ubiquitous:** Present everywhere in daily life; **reliability** and **scalability** are the two major design objectives.
- ❷ **Autonomic:** Can **self-organize** to support **dynamic discovery** of resources.
- ❸ **Composable:** Can be combined with guaranteed QoS (Quality of Service) and SLAs (Service-Level Agreements).
- ❹ Together, these paradigms and attributes realize the **computer utility vision**.

Common Characteristics of These Paradigms

- ❶ **Ubiquitous:** Present everywhere in daily life; **reliability** and **scalability** are the two major design objectives.
- ❷ **Autonomic:** Can **self-organize** to support **dynamic discovery** of resources.
- ❸ **Composable:** Can be combined with guaranteed **QoS** (Quality of Service) and **SLAs** (Service-Level Agreements).
- ❹ Together, these paradigms and attributes realize the **computer utility vision**.

Common Characteristics of These Paradigms

- ① **Ubiquitous:** Present everywhere in daily life; **reliability** and **scalability** are the two major design objectives.
- ② **Autonomic:** Can **self-organize** to support **dynamic discovery** of resources.
- ③ **Composable:** Can be combined with guaranteed **QoS** (Quality of Service) and **SLAs** (Service-Level Agreements).
- ④ Together, these paradigms and attributes realize the **computer utility vision**.

Utility Computing

A business model in which customers receive computing resources from a **paid service provider**, just like electricity or water.

- ① All grid and cloud platforms are regarded as **utility service providers**.
- ② Cloud computing is a **broader concept** than utility computing.
- ③ Distributed cloud applications run on any available servers in some edge networks.
- ④ **Technological challenges** include network-efficient processors, scalable memory and storage, distributed OSes, virtualization middleware, new programming models, and effective resource management.
- ⑤ **Analogy:** You pay the electricity bill for units consumed; you do not build a power plant at home.

Utility Computing

A business model in which customers receive computing resources from a **paid service provider**, just like electricity or water.

- ① All grid and cloud platforms are regarded as **utility service providers**.
- ② Cloud computing is a **broader concept** than utility computing.
- ③ Distributed cloud applications run on any available servers in some edge networks.
- ④ Technological challenges include network-efficient processors, scalable memory and storage, distributed OSes, virtualization middleware, new programming models, and effective resource management.
- ⑤ **Analogy:** You pay the electricity bill for units consumed; you do not build a power plant at home.

Utility Computing

A business model in which customers receive computing resources from a **paid service provider**, just like electricity or water.

- ① All grid and cloud platforms are regarded as **utility service providers**.
- ② Cloud computing is a **broadier concept** than utility computing.
- ③ Distributed cloud applications run on any available servers in some edge networks.
- ④ Technological challenges include network-efficient processors, scalable memory and storage, distributed OSES, virtualization middleware, new programming models, and effective resource management.
- ⑤ **Analogy:** You pay the electricity bill for units consumed; you do not build a power plant at home.

Utility Computing

A business model in which customers receive computing resources from a **paid service provider**, just like electricity or water.

- ① All grid and cloud platforms are regarded as **utility service providers**.
 - ② Cloud computing is a **broadier concept** than utility computing.
 - ③ Distributed cloud applications run on any available servers in some edge networks.
 - ④ **Technological challenges** include network-efficient processors, scalable memory and storage, distributed OSES, virtualization middleware, new programming models, and effective resource management.
- **Analogy:** You pay the electricity bill for units consumed; you do not build a power plant at home.

Utility Computing

A business model in which customers receive computing resources from a **paid service provider**, just like electricity or water.

- ① All grid and cloud platforms are regarded as **utility service providers**.
- ② Cloud computing is a **broadier concept** than utility computing.
- ③ Distributed cloud applications run on any available servers in some edge networks.
- ④ **Technological challenges** include network-efficient processors, scalable memory and storage, distributed OSES, virtualization middleware, new programming models, and effective resource management.
- ⑤ **Analogy:** You pay the electricity bill for units consumed; you do not build a power plant at home.

The Hype Cycle of New Technologies

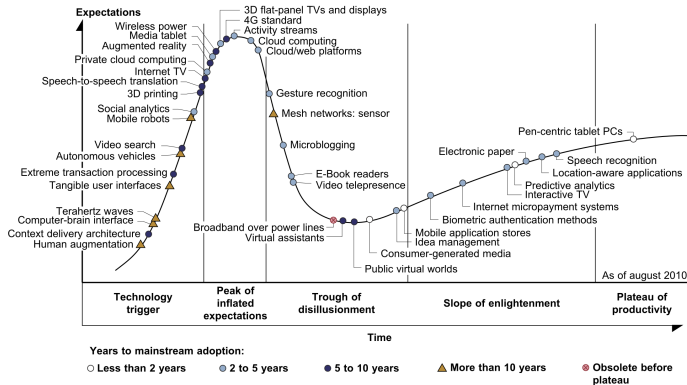


Figure: Hype cycle for emerging technologies, 2010 (Fig. 1.3; source: Gartner, Inc., 2010, as reproduced in Text Book 1)

Five Stages of the Hype Cycle

- ❶ **Technology trigger:** A new technology appears; expectations begin to rise.
- ❷ **Peak of inflated expectations:** Expectations rise sharply — often beyond reality.
- ❸ **Trough of disillusionment:** After a short period, expectations drop into a valley.
- ❹ **Slope of enlightenment:** Expectations increase steadily over a long period as real uses are understood.
- ❺ **Plateau of productivity:** The technology reaches **mainstream adoption**.
- ❻ Once a technology climbs the slope of enlightenment, it may reach the plateau within **two to five years**.

Five Stages of the Hype Cycle

- 1 **Technology trigger:** A new technology appears; expectations begin to rise.
 - 2 **Peak of inflated expectations:** Expectations rise sharply — often beyond reality.
 - 3 **Trough of disillusionment:** After a short period, expectations drop into a valley.
 - 4 **Slope of enlightenment:** Expectations increase steadily over a long period as real uses are understood.
 - 5 **Plateau of productivity:** The technology reaches mainstream adoption.
- Once a technology climbs the slope of enlightenment, it may reach the plateau within two to five years.

Five Stages of the Hype Cycle

- ❶ **Technology trigger:** A new technology appears; expectations begin to rise.
 - ❷ **Peak of inflated expectations:** Expectations rise sharply — often beyond reality.
 - ❸ **Trough of disillusionment:** After a short period, expectations drop into a valley.
 - ❹ **Slope of enlightenment:** Expectations increase steadily over a long period as real uses are understood.
 - ❺ **Plateau of productivity:** The technology reaches mainstream adoption.
- ❻ Once a technology climbs the slope of enlightenment, it may reach the plateau within two to five years.

Five Stages of the Hype Cycle

- ❶ **Technology trigger:** A new technology appears; expectations begin to rise.
- ❷ **Peak of inflated expectations:** Expectations rise sharply — often beyond reality.
- ❸ **Trough of disillusionment:** After a short period, expectations drop into a valley.
- ❹ **Slope of enlightenment:** Expectations increase steadily over a long period as real uses are understood.
- ❺ **Plateau of productivity:** The technology reaches mainstream adoption.
- ❻ Once a technology climbs the slope of enlightenment, it may reach the plateau within two to five years.

Five Stages of the Hype Cycle

- ❶ **Technology trigger:** A new technology appears; expectations begin to rise.
 - ❷ **Peak of inflated expectations:** Expectations rise sharply — often beyond reality.
 - ❸ **Trough of disillusionment:** After a short period, expectations drop into a valley.
 - ❹ **Slope of enlightenment:** Expectations increase steadily over a long period as real uses are understood.
 - ❺ **Plateau of productivity:** The technology reaches **mainstream adoption**.
- ❶ Once a technology climbs the slope of enlightenment, it may reach the plateau within two to five years.

Five Stages of the Hype Cycle

- ❶ **Technology trigger:** A new technology appears; expectations begin to rise.
- ❷ **Peak of inflated expectations:** Expectations rise sharply — often beyond reality.
- ❸ **Trough of disillusionment:** After a short period, expectations drop into a valley.
- ❹ **Slope of enlightenment:** Expectations increase steadily over a long period as real uses are understood.
- ❺ **Plateau of productivity:** The technology reaches **mainstream adoption**.
- ❻ Once a technology climbs the slope of enlightenment, it may reach the plateau within **two to five years**.

Reading the Hype Cycle: Symbols and Examples (2010)

Symbol	Years to Mainstream	Example (as of August 2010)
Hollow circle	Less than 2 years	Consumer-generated media (in the trough)
Gray circle	2 to 5 years	Internet micropayment systems; cloud computing
Solid (dark) circle	5 to 10 years	3D printing (rising expectations)
Triangle	More than 10 years	Mesh network sensors
Crossed circle	Obsolete before plateau	Broadband over power lines

- ❶ In 2010, **cloud technology** had just crossed the **peak** and was expected to reach productivity in **2–5 years**.
- ❷ Other promising technologies then: biometric authentication, interactive TV, speech recognition, predictive analytics, media tablets.
- ❸ **Think:** Where would you place cloud computing, AI chatbots, or 3D printing on the hype cycle today?

Reading the Hype Cycle: Symbols and Examples (2010)

Symbol	Years to Mainstream	Example (as of August 2010)
Hollow circle	Less than 2 years	Consumer-generated media (in the trough)
Gray circle	2 to 5 years	Internet micropayment systems; cloud computing
Solid (dark) circle	5 to 10 years	3D printing (rising expectations)
Triangle	More than 10 years	Mesh network sensors
Crossed circle	Obsolete before plateau	Broadband over power lines

- 1 In 2010, **cloud technology** had just crossed the **peak** and was expected to reach productivity in **2–5 years**.
- 2 Other promising technologies then: biometric authentication, interactive TV, speech recognition, predictive analytics, media tablets.
- 3 Think: Where would you place cloud computing, AI chatbots, or 3D printing on the hype cycle today?

Reading the Hype Cycle: Symbols and Examples (2010)

Symbol	Years to Mainstream	Example (as of August 2010)
Hollow circle	Less than 2 years	Consumer-generated media (in the trough)
Gray circle	2 to 5 years	Internet micropayment systems; cloud computing
Solid (dark) circle	5 to 10 years	3D printing (rising expectations)
Triangle	More than 10 years	Mesh network sensors
Crossed circle	Obsolete before plateau	Broadband over power lines

- 1 In 2010, **cloud technology** had just crossed the **peak** and was expected to reach productivity in **2–5 years**.
- 2 Other promising technologies then: biometric authentication, interactive TV, speech recognition, predictive analytics, media tablets.
- 3 **Think:** Where would you place cloud computing, AI chatbots, or 3D printing on the hype cycle **today**?

- ❶ Computing has evolved from **mainframes** to **clusters, grids, and Internet clouds** over five generations.
- ❷ HPC focuses on raw speed; HTC focuses on the number of tasks completed per unit time.
- ❸ SOA, virtualization, and RFID/GPS/sensors gave rise to Web 2.0 services, clouds, and the IoT.
- ❹ Centralized, parallel, distributed, and cloud computing differ in how resources and memory are shared.
- ❺ Parallelism has grown from bit-level to job-level.
- ❻ Modern paradigms are ubiquitous, autonomic, and composable, realizing the computer utility vision.
- ❼ New technologies follow a hype cycle before reaching mainstream adoption.

Summary

- ❶ Computing has evolved from **mainframes** to **clusters, grids, and Internet clouds** over five generations.
- ❷ **HPC** focuses on raw speed; **HTC** focuses on the number of tasks completed per unit time.
- ❸ SOA, virtualization, and RFID/GPS/sensors gave rise to Web 2.0 services, clouds, and the IoT.
- ❹ Centralized, parallel, distributed, and cloud computing differ in how resources and memory are shared.
- ❺ Parallelism has grown from bit-level to job-level.
- ❻ Modern paradigms are ubiquitous, autonomic, and composable, realizing the computer utility vision.
- ❼ New technologies follow a **hype cycle** before reaching mainstream adoption.

Summary

- 1 Computing has evolved from **mainframes** to **clusters, grids, and Internet clouds** over five generations.
- 2 **HPC** focuses on raw speed; **HTC** focuses on the number of tasks completed per unit time.
- 3 **SOA, virtualization, and RFID/GPS/sensors** gave rise to Web 2.0 services, clouds, and the IoT.
- 4 Centralized, parallel, distributed, and cloud computing differ in how resources and memory are shared.
- 5 Parallelism has grown from bit-level to job-level.
- 6 Modern paradigms are ubiquitous, autonomic, and composable, realizing the computer utility vision.
- 7 New technologies follow a hype cycle before reaching mainstream adoption.

- 1 Computing has evolved from **mainframes** to **clusters, grids, and Internet clouds** over five generations.
 - 2 **HPC** focuses on raw speed; **HTC** focuses on the number of tasks completed per unit time.
 - 3 **SOA, virtualization, and RFID/GPS/sensors** gave rise to Web 2.0 services, clouds, and the IoT.
 - 4 Centralized, parallel, distributed, and cloud computing differ in **how resources and memory are shared**.
- Parallelism has grown from **bit-level** to **job-level**.
 - Modern paradigms are **ubiquitous, autonomic, and composable**, realizing the **computer utility vision**.
 - New technologies follow a **hype cycle** before reaching mainstream adoption.

Summary

- 1 Computing has evolved from **mainframes** to **clusters, grids, and Internet clouds** over five generations.
 - 2 **HPC** focuses on raw speed; **HTC** focuses on the number of tasks completed per unit time.
 - 3 **SOA, virtualization, and RFID/GPS/sensors** gave rise to Web 2.0 services, clouds, and the IoT.
 - 4 Centralized, parallel, distributed, and cloud computing differ in **how resources and memory are shared**.
 - 5 Parallelism has grown from **bit-level** to **job-level**.
- Modern paradigms are **ubiquitous, autonomic, and composable**, realizing the computer utility vision.
 - New technologies follow a **hype cycle** before reaching mainstream adoption.

Summary

- ① Computing has evolved from **mainframes** to **clusters, grids, and Internet clouds** over five generations.
- ② **HPC** focuses on raw speed; **HTC** focuses on the number of tasks completed per unit time.
- ③ **SOA, virtualization, and RFID/GPS/sensors** gave rise to Web 2.0 services, clouds, and the IoT.
- ④ Centralized, parallel, distributed, and cloud computing differ in **how resources and memory are shared**.
- ⑤ Parallelism has grown from **bit-level** to **job-level**.
- ⑥ Modern paradigms are **ubiquitous, autonomic, and composable**, realizing the **computer utility** vision.
- ⑦ New technologies follow a **hype cycle** before reaching mainstream adoption.

- ① Computing has evolved from **mainframes** to **clusters, grids, and Internet clouds** over five generations.
- ② **HPC** focuses on raw speed; **HTC** focuses on the number of tasks completed per unit time.
- ③ **SOA, virtualization, and RFID/GPS/sensors** gave rise to Web 2.0 services, clouds, and the IoT.
- ④ Centralized, parallel, distributed, and cloud computing differ in **how resources and memory are shared**.
- ⑤ Parallelism has grown from **bit-level** to **job-level**.
- ⑥ Modern paradigms are **ubiquitous, autonomic, and composable**, realizing the **computer utility** vision.
- ⑦ New technologies follow a **hype cycle** before reaching mainstream adoption.

Review Questions

- 1 List the five generations of computer platforms with one example each.
- 2 Differentiate between HPC and HTC systems with suitable examples.
- 3 Explain the evolutionary trend shown in Figure 1.1.
- 4 Distinguish centralized, parallel, distributed, and cloud computing.
- 5 What are the three new computing paradigms? What enabled each of them?
- 6 Explain the five degrees of parallelism.
- 7 Explain the vision of computer utilities with a neat diagram (Figure 1.2).
- 8 Describe the five stages of the hype cycle of new technologies.